Bayesian Networks and Autoregressive Language Models: Lab Report

Course: CS F407 Artificial Intelligence. **Files:** `bn_lab.py` (first-order and second-order models), `bn_experiments.py` (all experiments), `generated_sentences.txt` (the saved generated text). Tokens are lower-case words with <START> and <END> markers; the 6-sentence dataset from the lab is used. Every number below was produced by running the code; sampling uses fixed seeds.

Part I-II: Questions 1 and 2

Q1. The chain rule turns the hard problem of modelling a whole sentence into a series of next-word problems: P(x1..xT) = P(x1) P(x2|x1) ... P(xT|x1..xT-1). Generation then becomes sampling one token at a time from a conditional distribution given what has been generated so far, and the probability of any complete sentence is the product of the conditionals, so the same model can both score and generate text.

Q2. The chain X1 -> X2 -> ... -> XT assumes that each word is independent of all earlier words given the immediately preceding word: P(Xt | X1,...,Xt-1) = P(Xt | Xt-1), i.e. Xt is conditionally independent of X1,...,Xt-2 given Xt-1 (the first-order Markov assumption).

Part IV: Question 3, first-order conditional probability tables

Counts come from the dataset (the word 'the' occurs 12 times, so P(cat | the) = 3/12; the 3/5 and 2/5 in the lab text are only an illustration).

Context w	P(next | w)	Zero-probability next tokens
the	cat 0.250, dog 0.250, mat 0.167, rug 0.167, park 0.167	on, ran, sat, the, to, <END>
cat	sat 0.667, ran 0.333	cat, dog, mat, on, park, rug, the, to, <END>
dog	sat 0.667, ran 0.333	cat, dog, mat, on, park, rug, the, to, <END>
sat	on 1.000	cat, dog, mat, park, ran, rug, sat, the, to, <END>
ran	to 1.000	cat, dog, mat, on, park, ran, rug, sat, the, <END>

Context w	P(next | w)
<START>	the 1.000
on	the 1.000
to	the 1.000
mat	<END> 1.000
rug	<END> 1.000
park	<END> 1.000

Zero-probability transitions are all next tokens not listed above, for example P(mat | cat) = 0, P(dog | sat) = 0, P(<END> | the) = 0; and a word such as 'on' can never be followed by anything but 'the'. Note the model nevertheless gives P(<END> | mat) = 1 and P(mat | the) = 1/6, so it will allow 'the mat' as a complete sentence.

Parts V-VII: LLM prompt, code inspection and tests (Questions 4-8)

[Personalise: replace with the exact prompt(s) you used]

```
Write a simple Python implementation of a first-order autoregressive language model.
It should: take tokenised sentences; count transitions between consecutive tokens;
construct P(Xt | Xt-1); display probabilities for a given previous token; predict the
most probable next token; generate a sentence by repeatedly sampling the next token;
stop when <END> is generated. Do not use any ML library or pretrained model; use
ordinary Python data structures and random sampling.
```

Q4. Transition counts are stored in `self.counts`, a dictionary mapping a context to a Counter of next tokens, filled in `fit()`. **Q5.** P(Xt | Xt-1) is computed in `dist()`: each count divided by the total count for that context. **Q6.** The program supports both. `generate(mode='sample')` draws the next word from the distribution with

`rng.choices` weighted by probability (default); `mode='greedy'` always takes the arg max. Greedy is deterministic and always repeats the most likely continuation; sampling picks a word with probability equal to its model probability, so less likely words still appear sometimes and the output varies. **Q7.** For an unseen context `dist()` returns an empty distribution (e.g. dist(['banana']) = {} and predict returns None) instead of crashing; `generate()` then stops. This is a deliberate choice: the model has no information, so it cannot say anything (smoothing or back-off would be needed to do better).

Normalisation test (Part VII). For each context the program sums its distribution: first-order, 11 contexts, every total equals 1.0 (largest deviation 0.0e+00); second-order, 15 contexts, every total equals 1.0 (largest deviation 0.0e+00). **Q8.** A total of 0.87 would mean 13% of the probability mass is missing: the probabilities are not a valid distribution. Likely causes are normalising by the wrong denominator (e.g. leaving out transitions to <END>), dropping some tokens or counts, or an incorrect smoothing step. Sampling from it would be biased or fail.

Part VIII: Next-word prediction (Question 9)

Context	Distribution	arg max
<START>	the 1.000	the
the	cat 0.250, dog 0.250, mat 0.167, rug 0.167, park 0.167	cat / dog (tie)
cat	sat 0.667, ran 0.333	sat
sat	on 1.000	on
ran	to 1.000	to
on	the 1.000	the
to	the 1.000	the

Q9. Not always. Ties and context-blindness show up: after 'the' the model rates cat and dog equally (0.25 each) and gives 'mat', 'rug' or 'park' 0.5 in total, so it will happily produce 'the rug'. A person who has seen 'sat on' would expect mat/rug, but the first-order model forgets everything but the last word. A probability model reflects counts and its independence assumption in the training data; human expectation uses grammar, meaning and the whole sentence so far.

Parts IX-X: Generation and greedy versus sampling (Question 10)

20 sentences from each model are saved in `generated_sentences.txt` (seed 0). First 8 of each:

First-order (sampled)	Second-order (sampled)
the dog sat on the mat	the dog sat on the mat
the rug	the cat sat on the mat
the rug	the cat ran to the park
the dog sat on the dog ran to the park	the dog ran to the park
the cat sat on the park	the dog sat on the rug
the mat	the dog sat on the mat
the cat ran to the dog sat on the cat sat on the rug	the cat sat on the rug
the dog sat on the rug	the dog sat on the rug

Greedy vs sampling (5 sentences each, first-order):

Mode A: greedy	Mode B: sampling (seed 5)
the cat sat on the cat sat on the cat sat on ... (cycles; cut at 30 tokens)	the dog ran to the park
(identical)	the park
(identical)	the cat sat on the rug
(identical)	the mat
(identical)	the cat ran to the cat sat on the cat ran to the park

Second-order greedy always gives: 'the cat sat on the mat'; second-order sampling (seed 5) gives: the dog ran to the park; the dog sat on the mat; the dog sat on the rug; the dog sat on the mat; the cat ran to the park.

Q10. Sampling produces far more variation: greedy yields the same sentence every time (all five identical). For the first-order model greedy never even terminates, because arg max follows the cycle the -> cat -> sat -> on -> the -> cat ..., so a maximum length (30 tokens) is needed. Greedy removes randomness by always taking the single most probable word; sampling chooses each word in proportion to its probability.

Parts XI-XIII: Second-order model and comparison (Questions 11-12)

Selected second-order tables, P(Xt | Xt-2, Xt-1) (the first context pads with <START>):

Context (Xt-2, Xt-1)	P(next | context)
(<START>, <START>)	the 1.000
(<START>, the)	cat 0.500, dog 0.500
(the, cat)	sat 0.667, ran 0.333
(the, dog)	sat 0.667, ran 0.333
(cat, sat)	on 1.000
(sat, on)	the 1.000
(on, the)	mat 0.500, rug 0.500
(cat, ran)	to 1.000
(ran, to)	the 1.000
(to, the)	park 1.000
(the, mat)	<END> 1.000

Measure	First-order	Second-order
Distinct parameters (non-zero P entries)	17	19
Contexts observed / possible	11 / 11	15 / 121
Contexts never seen (no distribution at all)	0	106
Full CPT size (contexts x 11 next tokens) and zero cells	121 cells, 104 zero	1331 cells, 1312 zero
Distinct sentences in 5000 samples	477	6
Samples identical to a training sentence	14.0%	100.0%
Samples hitting the 30-token cap	0.8%	0.0%
Average sentence length (tokens)	5.91	6.00

Qualitative examples. First-order output is varied but often wrong: 'the rug', 'the park', 'the cat sat on the park', and long loops such as 'the cat sat on the dog sat on the cat sat on the dog sat on the rug'. Second-order output is always fluent and sensible ('the dog ran to the park', 'the cat sat on the rug') but contains nothing new: it can only produce the 6 training sentences.

Q11. (1) Graph: first-order has one parent per node (Xt-1 -> Xt); second-order has two (Xt-2 -> Xt <- Xt-1). (2) CPT: rows indexed by one previous word (11 contexts) versus pairs of words (121 possible contexts), so the table is 11 times larger in rows. (3) Context: one word versus two, so the second-order model can distinguish 'sat on the' (mat/rug) from 'ran to the' (park). (4) Data: far more data is needed, because each context must be seen several times to estimate its row; here 106 of the 121 possible contexts were never observed.

Q12. More context carries more information about what comes next, which sharpens the prediction (after 'on the' it is only mat or rug, never cat). But the CPT has |V|^n rows, so the number of contexts grows exponentially with n while the amount of data stays fixed; each row is estimated from few examples, many are empty, and the model degenerates into memorising the training text (here: only 6 distinct sentences, no novelty).

Parts XIV-XV: Connection to modern models and role of the LLM (Questions 13-14)

Q13. Approach B is preferable because (i) it specifies the intended behaviour so the result can be judged against it; (ii) it fixes the representation (counts, CPT, sampling), which I must understand to interpret results; (iii) it can be validated: I tested that the code matches the specification; (iv) it exposes probabilistic invariants such as 'each row sums to 1' that can be tested automatically; and (v) it separates the model (the conditional distribution P(Xt | context)) from its implementation (dictionaries, sampling), so a bug in the implementation can be told apart from a limitation of the model. Approach A leaves all of this to the LLM's guess.

Q14. The Bayesian-network view gave me: **a factorisation of the joint distribution** (the product of one CPT per node), which tells me exactly what parameters must be estimated; **a way to reason about independence assumptions** (Q2: first-order drops all context but the last word) and thus to see why 'the rug' appears; **a principled method for generation** (ancestral sampling in order along the chain); **a way to understand the effect of increasing context** (more parents means a larger CPT, Q11-Q12); and **a way to test an implementation against its specification** (each CPT row must be a distribution summing to 1).

LLM use and validation

[Personalise: describe your own prompts and what you changed; the example below is a genuine finding from testing this code]

Inspected/corrected example (a design problem that testing exposes): in greedy mode the first-order model never reaches <END>, because arg max follows the cycle the -> cat -> sat -> on -> the ... forever. Greedy generation stopped only because `generate()` has a `max_len` guard that returns a 'truncated' flag; LLM-generated code that omits such a guard would hang here. Other checks I made: unseen contexts return an empty distribution instead of crashing; arg max ties are broken by first occurrence in the training data (cat before dog); and normalisation holds for every context in both models. Validation that the model matches its specification rests on these tests, not on the code looking plausible.